

AI Code Explanation Request



Please explain code i will give of ai subject also give me 20 questions on it for viva

Sure—send me the AI code, and I'll break it down in a viva-friendly way.

For each code, I can give you:

- ✓ **Purpose of the program** (what it does)
- ✓ **Line-by-line explanation** (simple words)
- ✓ **Algorithm / logic used**
- ✓ **Important AI concepts involved** (search, heuristic, ML, NLP, expert system, etc.)
- ✓ **Dry run / sample output explanation**
- ✓ **20 viva questions with answers** (from easy → tricky)
- ✓ **Possible modifications examiner may ask**
- ✓ **Short 30-second viva explanation** (so you can speak confidently)

Just paste your code here (Python / Java / any AI subject code), and I'll start.



```
from collections import deque
```

```
# Input graph from user
graph = {}
n = int(input("Enter number of edges: "))
print("Enter each edge as two space-separated nodes (e.g., A B):")
for _ in range(n):
    u, v = input().split()
    if u not in graph:
        graph[u] = []
    if v not in graph:
        graph[v] = []
    graph[u].append(v)
    graph[v].append(u) # Since it's an undirected graph

# Depth First Search (Recursive)
def dfs(graph, node, visited):
    if node not in visited:
        print(node, end=' ')
        visited.add(node)
        for neighbor in graph[node]:
            dfs(graph, neighbor, visited)
```

```
# Breadth First Search (Using Queue)
def bfs(graph, start, visited):
    queue = deque([start])
    while queue:
        node = queue.popleft()
        if node not in visited:
            print(node, end=' ')
            visited.add(node)
            queue.extend(graph[node])

# Main
start_node = input("Enter the starting node: ")

print("DFS Traversal:")
dfs(graph, start_node, set())

print("\nBFS Traversal:")
bfs(graph, start_node, set())
```

This is a classic **AI Search Algorithms program** implementing:

1. **DFS (Depth First Search)**
2. **BFS (Breadth First Search)**

These are fundamental in **Artificial Intelligence** for **state-space search / graph traversal**.

I'll explain it as if you're preparing for viva.

1) Purpose of Program

This program:

- Takes a **graph input** from user
- Stores graph using **Adjacency List**
- Performs:
 - **DFS traversal**
 - **BFS traversal**
- Prints nodes in traversal order

Used in AI for:

- Path finding
- Maze solving
- Game trees
- Web crawling

- Robot navigation
- State space search

2) Full Code Explanation

Python

```
from collections import deque
```

Meaning:

Imports **deque** from Python collections module.

deque = **Double Ended Queue**

Used for BFS because:

- insertion at end → fast
- deletion from front → fast

Normal list is slower.

Graph Creation

Python

```
graph = {}
```

Creates empty dictionary.

Example:

Python

```
{
'A': ['B', 'C'],
'B': ['A', 'D'],
'C': ['A'],
'D': ['B']
}
```

This is called **Adjacency List Representation**.

Python

```
n = int(input("Enter number of edges: "))
```

User enters number of connections.

Example:

If graph has:

A-B

A-C

B-D

Then:

`n = 3`

Python

```
print("Enter each edge as two space-separated nodes (e.g., A B):")
```

Instruction for user.

Loop for edges

Python

```
for _ in range(n):
```

Runs `n` times.

`_` means variable not needed.

Python

```
u, v = input().split()
```

Input:

A B

Then:

`u = A`

`v = B`

Python

```
if u not in graph:
    graph[u] = []
```

If node not present:

create empty list.

Example:

Python

```
graph['A'] = []
```

Python

```
if v not in graph:
    graph[v] = []
```

Same for second node.

Python

```
graph[u].append(v)
```

Add edge:

$u \rightarrow v$

Example:

$A \rightarrow B$

Python

```
graph[v].append(u)
```

Add reverse edge.

So:

$B \rightarrow A$

This makes graph **undirected**.

Final:

Python

```
A:[B]
```

```
B:[A]
```

Example Input

Suppose:

Edges:

A B

A C

B D

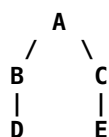
C E

Graph becomes:

Python

```
{
'A': ['B', 'C'],
'B': ['A', 'D'],
'C': ['A', 'E'],
'D': ['B'],
'E': ['C']
}
```

Visualization:



3) DFS Explanation

Code:

Python

```
def dfs(graph, node, visited):
```

Function for DFS.

Parameters:

graph → graph dictionary
node → current node
visited → set of visited nodes

Python

```
if node not in visited:
```

Checks if already visited.

Avoids infinite loop.

Example:

A→B→A→B→A

cycle forever.

Visited prevents this.

Python

```
print(node, end=' ')
```

Print current node.

Python

```
visited.add(node)
```

Mark visited.

Example:

Python

```
{'A', 'B'}
```

Python

```
for neighbor in graph[node]:
```

Visit all neighbors.

Example:

If:

Python

```
A: ['B', 'C']
```

neighbors:

B then C

Python

```
dfs(graph, neighbor, visited)
```

Recursive call.

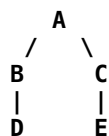
This goes deeper.

Hence:

Depth First Search.

DFS Working

Graph:



Start A

Visit:

A → B → D → backtrack → C → E

Output:

A B D C E

DFS Uses in AI

Used in:

- Puzzle solving
- Game trees
- Backtracking
- Constraint satisfaction
- Maze traversal

4) BFS Explanation

Code:

Python

```
def bfs(graph, start, visited):
```

BFS function.

Python

```
queue = deque([start])
```

Create queue.

Example:

Python

```
[A]
```

While queue not empty

Python

```
while queue:
```

Runs until queue empty.

Python

```
node = queue.popleft()
```

Remove front node.

FIFO:

First In First Out

Python

```
if node not in visited:
```

Avoid revisiting.

Python

```
print(node, end=' ')
```

Print node.

Python

```
visited.add(node)
```

Mark visited.

Python

```
queue.extend(graph[node])
```

Add neighbors.

Example:

A has:

B,C

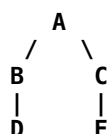
Queue becomes:

Python

```
[B,C]
```

BFS Working

Graph:



Start A

Queue:

[A]

Visit A → add B,C

Queue:

[B,C]

Visit B → add D

Queue:

[C,D]

Visit C → add E

Queue:

[D,E]

Visit D

Visit E

Output:

A B C D E

BFS Uses in AI

Used in:

- Shortest path
- Social networks
- Robot movement
- GPS routing
- Web crawling
- Knowledge graphs

5) Main Code

Python

```
start_node = input("Enter the starting node: ")
```

Input starting node.

Example:

A

Python

```
print("DFS Traversal:")
dfs(graph, start_node, set())
```

Run DFS.

Empty visited set passed.

Python

```
print("\nBFS Traversal:")
bfs(graph, start_node, set())
```

Run BFS.

6) Dry Run

Input:

```
4
A B
A C
B D
C E
A
```

DFS:

A B D C E

BFS:

A B C D E

7) AI Concepts Used

Important viva terms:

- State Space Search
 - Blind Search / Uninformed Search
 - DFS
 - BFS
 - Node Expansion
 - Queue
 - Stack (recursion stack)
 - Visited Set
 - Graph Traversal
 - Search Tree
-

8) Time Complexity

DFS:

O(V+E)

BFS:

$O(V+E)$

Where:

V = vertices

E = edges

9) 20 Viva Questions + Answers

Basic

1) What is BFS?

Breadth First Search explores level by level.

2) What is DFS?

Depth First Search explores deeply first.

3) Which data structure BFS uses?

Queue.

4) Which data structure DFS uses?

Stack / recursion stack.

5) Why visited set is used?

To avoid cycles/repeated visiting.

Medium

6) Is BFS informed search?

No, uninformed search.

7) Is DFS recursive?

Usually yes, but iterative possible.

8) BFS finds shortest path?

Yes in unweighted graph.

9) DFS finds shortest path?

No guarantee.

10) BFS memory requirement?

High.

11) DFS memory requirement?

Low.

12) Which is faster for deep solution?

DFS.

13) Which is better for shallow solution?

BFS.

14) What is adjacency list?

Graph representation using lists.

15) Graph type here?

Undirected graph.

Advanced

16) BFS complexity?

$O(V+E)$

17) DFS complexity?

$O(V+E)$

18) Can DFS get stuck?

Yes in deep infinite branch.

19) Can BFS handle cycles?

Yes using visited set.

20) AI applications?

Pathfinding, robotics, search problems, web crawling.

10) 30 Second Viva Answer

You can say:

"This Python program implements BFS and DFS graph traversal algorithms. Graph is stored using adjacency list. DFS uses recursion and explores depth-wise, while BFS uses queue and

explores level-wise. Visited set avoids revisiting nodes. In AI, these are uninformed search algorithms used in pathfinding, robot navigation, web crawling, and state-space search."

Perfect viva answer.

Send the next AI code when ready.



```

☐
import heapq

# User input for graph
graph = {}
n = int(input("Enter number of edges: "))
print("Enter each edge as: Node1 Node2 Cost")

for _ in range(n):
    u, v, cost = input().split()
    cost = int(cost)

    if u not in graph:
        graph[u] = []
    if v not in graph:
        graph[v] = []

    graph[u].append((v, cost))
    graph[v].append((u, cost)) # undirected graph

# User input for heuristics
heuristic = {}
nodes = set(graph.keys())
print("\nEnter heuristic values for each node:")

for node in nodes:
    h = int(input(f"Heuristic for {node}: "))
    heuristic[node] = h

# A* Algorithm
def astar(graph, start, goal, heuristic):
    open_set = [(heuristic[start], 0, start, [start])] # (f, g, node, path)
    visited = set()

    while open_set:
        f, g, current, path = heapq.heappop(open_set)

        if current == goal:
            return path, g

```

```

if current in visited:
    continue
visited.add(current)

for neighbor, cost in graph.get(current, []):
    if neighbor not in visited:
        new_g = g + cost
        new_f = new_g + heuristic[neighbor]
        heapq.heappush(open_set, (new_f, new_g, neighbor, path + [neighbor]))

return None, None

# Start and goal input
start = input("\nEnter start node: ")
goal = input("Enter goal node: ")

# Run A*
path, total_cost = astar(graph, start, goal, heuristic)

# Output
print()
if path:
    print("Shortest path from", start, "to", goal, ":", " -> ".join(path))
    print("Total cost:", total_cost)
else:
    print("No path found.")

```

This is an **A* (A-Star) Search Algorithm** program—one of the most important AI algorithms for viva. It is smarter than BFS/DFS because it uses:

- **Actual path cost** $\rightarrow g(n)$
- **Heuristic estimate** $\rightarrow h(n)$
- **Evaluation function** $\rightarrow f(n) = g(n) + h(n)$

This helps it find the goal **faster and optimally** (when heuristic is admissible).

1) Purpose of Program

This program:

- Takes weighted graph input
- Takes heuristic values
- Takes start node and goal node
- Applies **A* Search**
- Finds **best path**
- Calculates **minimum total cost**

Applications:

- Google Maps navigation
 - GPS routing
 - Robot path planning
 - Game AI
 - Maze solving
 - Network routing
-

2) Core AI Concept

A* chooses node with lowest:

$$f(n) = g(n) + h(n)$$

Where:

g(n)

Actual cost from start to current node

Example:

$A \rightarrow B \rightarrow C$

Cost:

$$2 + 3 = 5$$

So:

$$g(C)=5$$

h(n)

Estimated cost from current node to goal

This is heuristic.

Example:

Straight-line distance.

If estimate from C to Goal = 4

Then:

$$h(C)=4$$

f(n)

Total estimated cost

Example:

$g=5$

$h=4$

Then:

$f=9$

A* chooses node with smallest f .

3) Code Explanation

Import heapq

Python

```
import heapq
```

Used for **Priority Queue / Min Heap**

Why?

Because A* always picks lowest $f(n)$.

Heap makes that efficient.

Smallest value comes first.

Example:

```
[(5,A),(3,B),(8,C)]
```

Pop:

B first.

4) Graph Input

Python

```
graph = {}
```

Empty dictionary.

Will store weighted graph.

Example:

Python

```
{
    'A': [('B',2),('C',4)],
    'B': [('A',2),('D',5)]
}
```

Python

```
n = int(input("Enter number of edges: "))
```

Input number of edges.

Python

```
u, v, cost = input().split()
```

Printed using [ChatGPT to PDF](#), powered by PDFCrowd [HTML to PDF API](#).

Input:

A B 3

Means:

A connected to B with cost 3.

Python

```
cost = int(cost)
```

Convert string to integer.

Create nodes if absent

Python

```
if u not in graph:
    graph[u] = []
```

Create empty adjacency list.

Same for v.

Add edge

Python

```
graph[u].append((v, cost))
graph[v].append((u, cost))
```

Undirected weighted graph.

Example:

A-B cost 5

Stored:

Python

```
A: [(B, 5)]
B: [(A, 5)]
```

5) Heuristic Input

Python

```
heuristic = {}
```

Stores h(n).

Example:

Python

```
{
    'A': 10,
    'B': 8,
    'C': 5,
```

```
'D': 0  
}
```

Goal usually heuristic 0.

Python

```
nodes = set(graph.keys())
```

Get all nodes.

Python

```
for node in nodes:
```

Ask heuristic for each.

Python

```
heuristic[node] = h
```

Store value.

6) A* Function

Python

```
def astar(graph, start, goal, heuristic):
```

Function definition.

Parameters:

- graph
 - start
 - goal
 - heuristic
-

Open Set

Python

```
open_set = [(heuristic[start], 0, start, [start])]
```

Stores:

(f,g,node,path)

Initially:

For A:

Suppose:

$h(A)=10$

Then:

```
(10,0,'A',['A'])
```

Meaning:

f=10

g=0

current=A

path=[A]

Visited

Python

```
visited = set()
```

Avoid revisiting.

Main loop

Python

```
while open_set:
```

Until heap empty.

Remove best node

Python

```
f, g, current, path = heapq.heappop(open_set)
```

Removes minimum f.

This is heart of A*.

Goal check

Python

```
if current == goal:  
    return path, g
```

Goal reached.

Return:

- path
 - total cost
-

Skip duplicates

Python

```
if current in visited:
    continue
```

Already processed → skip.

Python

```
visited.add(current)
```

Mark visited.

Expand neighbors

Python

```
for neighbor, cost in graph.get(current, []):
```

Visit connected nodes.

Compute new g

Python

```
new_g = g + cost
```

Actual cost increases.

Example:

$A \rightarrow B = 4$

Current $g=2$

$new_g=6$

Compute new f

Python

```
new_f = new_g + heuristic[neighbor]
```

Evaluation.

Example:

$g=6$

$h=3$

$f=9$

Push into heap

Python

```
heapq.heappush(open_set, (new_f, new_g, neighbor, path+[neighbor]))
```

Printed using [ChatGPT to PDF](#), powered by PDFCrowd [HTML to PDF API](#).

Insert candidate.

Heap sorts automatically.

Lowest f on top.

No path

Python

`return None, None`

If unreachable.

7) Example Dry Run

Graph:

```

A--2--B
|    / |
4    3  5
|    / |
C--1--D

```

Heuristic:

```

A=7
B=4
C=2
D=0

```

Goal = D

Start:

```

A
f=7
g=0

```

Expand:

B:

```

g=2
f=2+4=6

```

C:

```

g=4
f=4+2=6

```

Choose one minimum.

Suppose B.

Expand B:

D:

$g=7$

$f=7+0=7$

C via B:

$g=5$

$f=5+2=7$

Then explore C:

D:

$g=5$

$f=5$

Best path:

$A \rightarrow C \rightarrow D$

Cost = 5

8) Why A* Is Better

Compared to BFS:

BFS explores blindly.

Compared to DFS:

DFS may go wrong direction.

Compared to Dijkstra:

Dijkstra uses only cost.

A* uses:

Cost + intelligence.

So faster.

9) Complexity

Worst case:

Time:

$O(E \log V)$

Space:

High (stores many nodes)

10) Important Terms for Viva

Say these confidently:

- Heuristic Search

- Best First Search
 - Evaluation Function
 - Admissible Heuristic
 - Optimal Path
 - Priority Queue
 - Open Set
 - Closed Set / Visited Set
 - Path Cost
 - Goal State
-

11) 20 Viva Questions + Answers

Basic

1) What is A* algorithm?

Heuristic search algorithm for shortest optimal path.

2) Formula used?

Uses:

$$f(n)=g(n)+h(n)$$

3) What is g(n)?

Actual cost from start node.

4) What is h(n)?

Estimated cost to goal.

5) What is f(n)?

Total estimated cost.

6) Which data structure is used?

Priority Queue / Heap.

7) Which module used?

heapq

8) Is A* informed search?

Yes.

9) Why informed?

Uses heuristic knowledge.

10) Goal heuristic value?

Usually 0.

Medium

11) Is A* optimal?

Yes, if heuristic admissible.

12) Admissible heuristic means?

Never overestimates true cost.

13) Example heuristic?

Straight line distance.

14) Can A* fail?

If no path exists.

15) Difference A* vs Dijkstra?

A* uses heuristic.

Dijkstra doesn't.

Advanced

16) What is open set?

Nodes waiting exploration.

17) Closed set?

Visited nodes.

18) Complexity?

$O(E \log V)$

19) AI applications?

Robotics, GPS, games.

20) Why better than BFS?

Guided search, less exploration.

12) 30 Second Viva Answer

Say:

"This program implements A* search algorithm on a weighted graph. It uses heuristic values and path cost to calculate $f(n)=g(n)+h(n)$. The node with minimum f value is explored first using priority queue. This makes A* an informed search algorithm and helps find optimal path efficiently. It is widely used in GPS routing, robotics, and game AI."

Perfect viva answer.

This is a **very important AI practical**. Send the next code.



```

☐
def find(p, i):
    while p[i] != i:
        i = p[i]
    return i

def union(p, x, y):
    p[find(p, y)] = find(p, x)

# Input
n = int(input("Enter number of vertices: "))
e = int(input("Enter number of edges: "))
edges = []

print("Enter edges (u v weight):")
for _ in range(e):
    u, v, w = map(int, input().split())
    edges.append((w, u, v))

edges.sort()
parent = [i for i in range(n)]
mst = []

for w, u, v in edges:
    if find(parent, u) != find(parent, v):
        mst.append((u, v, w))
        union(parent, u, v)

# Output
print("\nMinimum Spanning Tree:")
tc=0
for u, v, w in mst:
    tc+=w
  
```

```
print(f"{u} --> {v} == {w}")
print(tc)
```

This program implements **Kruskal's Algorithm**, a very important **Greedy Algorithm in AI + Data Structures**, used to find the **Minimum Spanning Tree (MST)** of a graph.

It is commonly asked in **viva + practical exams**.

1) Purpose of Program

This program:

- Takes a weighted graph as input
- Sorts edges by weight
- Uses **Union-Find (Disjoint Set)** structure
- Builds **Minimum Spanning Tree (MST)**
- Prints:
 - Selected edges
 - Total cost

2) Key Concept (AI + DS)

MST means:

- 👉 Connect all vertices
- 👉 With minimum total cost
- 👉 Without cycles

Used in:

- Network design (LAN, internet)
- Road construction
- Electrical circuits
- Clustering in AI
- Optimization problems

3) Important Idea (Greedy Approach)

Kruskal works using:

- ✓ Always pick smallest edge first
- ✓ Avoid cycles

✓ Stop when all nodes connected

4) Code Explanation

◆ FIND FUNCTION (Root finding)

Python

```
def find(p, i):
    while p[i] != i:
        i = p[i]
    return i
```

Meaning:

Finds **representative (root)** of a node.

Example:

0 → 1 → 2

Root = 2

Why needed?

To check:

👉 Are two nodes in same set?

If yes → skip edge (cycle)

◆ UNION FUNCTION

Python

```
def union(p, x, y):
    p[find(p, y)] = find(p, x)
```

Meaning:

Connect two components.

Example:

A and B are separate sets → merge them.

5) Input Section

Python

```
n = int(input("Enter number of vertices: "))
e = int(input("Enter number of edges: "))
```

- n = vertices
- e = edges

Python

```
edges = []
```

Stores all edges.

Python

```
u, v, w = map(int, input().split())
```

Input format:

0 1 4

Means:

$0 \rightarrow 1$ with weight 4

Python

```
edges.append((w, u, v))
```

Stored as:

👉 (weight, u, v)

Reason:

Sorting becomes easy.

6) Sorting Edges

Python

```
edges.sort()
```

Sort by weight (ascending)

Example:

```
(1,0,1)
(2,1,2)
(5,0,2)
```

7) Parent Array

Python

```
parent = [i for i in range(n)]
```

Initially:

Each node is its own parent.

Example:

```
[0,1,2,3]
```

Meaning:

Each node is separate set.

8) MST List

Python
`mst = []`

Stores selected edges.

9) Main Kruskal Loop

Python
`for w, u, v in edges:`

Process edges from smallest weight.

Cycle Check

Python
`if find(parent, u) != find(parent, v):`

If roots are different:

- ✓ No cycle
 - ✓ Safe to add edge
-

Add to MST

Python
`mst.append((u, v, w))`

Edge included.

Union Operation

Python
`union(parent, u, v)`

Merge sets.

10) Output Section

Python
`print("\nMinimum Spanning Tree:")`

Print edges

Python

```
for u, v, w in mst:
    tc += w
    print(f"{u} --> {v} == {w}")
```

- Displays MST edges
- Calculates total cost

Total Cost

Python

```
print(tc)
```

Final MST weight.

11) Example Dry Run

Input:

```
Vertices = 4
Edges = 5
0 1 10
0 2 6
0 3 5
1 3 15
2 3 4
```

Sorted edges:

```
2-3 = 4
0-3 = 5
0-2 = 6
0-1 = 10
1-3 = 15
```

Kruskal picks:

```
✓ 2-3 (4)
✓ 0-3 (5)
✗ 0-2 (cycle skip)
✓ 0-1 (10)
```

MST:

```
2 --> 3 == 4
0 --> 3 == 5
0 --> 1 == 10
```

Total cost = 19

12) AI Concepts Used

- Greedy Search Strategy
 - Optimization Problem
 - Graph Theory
 - Disjoint Set (Union-Find)
 - Minimum Spanning Tree
 - Cycle Detection
-

13) Time Complexity

Main cost:

Sorting edges $\rightarrow O(E \log E)$

Union-Find $\rightarrow$ nearly $O(1)$

Total:

👉 $O(E \log E)$

14) Important Viva Questions (20)

Basic

1) What is Kruskal's algorithm?

Greedy algorithm to find Minimum Spanning Tree.

2) What is MST?

Tree connecting all vertices with minimum cost.

3) Which data structure used?

Union-Find (Disjoint Set)

4) What is greedy approach?

Chooses best option at each step.

5) Why edges are sorted?

To pick minimum weight first.

Medium

6) What is cycle detection method?

Using Union-Find.

7) Why MST has no cycles?

Because cycle edges are skipped.

8) Time complexity?

$O(E \log E)$

9) Difference between Kruskal and Prim?

Kruskal → edges

Prim → nodes

10) What is parent array?

Stores representative of set.

Advanced

11) What is find() function?

Finds root parent of node.

12) What is union() function?

Merges two sets.

13) What is disjoint set?

Collection of non-overlapping sets.

14) Why used in AI?

Optimization and clustering problems.

15) Can Kruskal fail?

No, if graph is connected.

Expert

16) Is Kruskal optimal?

Yes, gives minimum cost MST.

17) Is it greedy or dynamic?

Greedy algorithm.

18) What if graph disconnected?

Forest (multiple MSTs).

19) What is edge sorting used for?

To ensure greedy selection.

20) Real applications?

Network design, clustering, routing.

15) 30-Second Viva Answer

Say this:

"This program implements Kruskal's algorithm to find Minimum Spanning Tree using a greedy approach. It sorts edges by weight and uses Union-Find to detect cycles. If two nodes belong to different sets, the edge is added to MST. Finally, it prints all selected edges and total cost. It is widely used in network design and optimization problems."

If you want next level preparation, I can also:

- ✓ Compare BFS vs DFS vs A* vs Kruskal (VERY IMPORTANT FOR VIVA)
- ✓ Give 1-page revision sheet
- ✓ Or give "most repeated viva questions from AI practicals"

Just tell 👍



```

import heapq

def prim_mst(graph, start=0):
    mst = []
    visited = [False] * len(graph)
    min_heap = [(0, start)]
    total_weight = 0

    while min_heap:
        weight, u = heapq.heappop(min_heap)

        if visited[u]:
            continue
        visited[u] = True
        total_weight += weight

        for v, w in graph[u]:
            if not visited[v]:

```

```

        heapq.heappush(min_heap, (w, v))
        mst.append((u, v, w))

    return mst, total_weight

# ♦ Dynamic input
n = int(input("Enter number of vertices: "))
e = int(input("Enter number of edges: "))

graph = {i: [] for i in range(n)}
print("Enter each edge in the format: u v w (0-based indexing)")
for _ in range(e):
    u, v, w = map(int, input().split())
    graph[u].append((v, w))
    graph[v].append((u, w)) # Undirected graph

mst, weight = prim_mst(graph)
print("\nPrim's MST edges:")
for u, v, w in mst:
    print(f"{u} -- {v} == {w}")
print("Total weight of MST:", weight)

```

This is **Prim's Algorithm**, another very important **Greedy Algorithm in AI + Graph Theory**, used to find the **Minimum Spanning Tree (MST)**.

It is heavily asked in **viva exams**, especially in comparison with Kruskal.

1) Purpose of Program

This program:

- Takes weighted undirected graph input
- Uses **Prim's Algorithm**
- Builds MST starting from a node
- Uses **Min Heap (Priority Queue)**
- Outputs:
 - MST edges
 - Total weight

2) Core Idea of Prim's Algorithm

Prim works like:

- 👉 Start from any node
- 👉 Always pick **minimum weight edge** connecting visited → unvisited
- 👉 Grow tree step by step

It is **greedy + node-based expansion**

3) Key Formula (Conceptual)

Prim does NOT use formula like A*

But logic is:

- 👉 Always choose **minimum edge crossing cut**

4) Code Explanation

◆ Import heap

Python

```
import heapq
```

Used for **priority queue (min heap)**

- ✓ Always extracts smallest edge first
- ✓ Very important for Prim

5) Function Definition

Python

```
def prim_mst(graph, start=0):
```

- graph = adjacency list
- start = starting node

6) MST Storage

Python

```
mst = []
```

Stores selected edges.

7) Visited Array

Python

```
visited = [False] * len(graph)
```

Tracks visited nodes.

Example:

```
[False, False, False]
```

8) Min Heap Initialization

Python

```
min_heap = [(0, start)]
```

Format:

```
(weight, node)
```

Start node has weight 0.

9) Total Weight

Python

```
total_weight = 0
```

Stores MST cost.

10) Main Loop

Python

```
while min_heap:
```

Runs until heap empty.

Extract Minimum Edge

Python

```
weight, u = heapq.heappop(min_heap)
```

✓ Picks smallest weight node

Skip if already visited

Python

```
if visited[u]:  
    continue
```

Avoid duplicates.

Mark visited

Python

```
visited[u] = True
```

Node included in MST.

Add weight

Python

```
total_weight += weight
```

Add edge cost.

11) Explore neighbors

Python

```
for v, w in graph[u]:
```

Go through adjacency list.

If not visited

Python

```
if not visited[v]:
```

Check unvisited node.

Push into heap

Python

```
heapq.heappush(min_heap, (w, v))
```

Add edge to priority queue.

Store MST edge

Python

```
mst.append((u, v, w))
```

Add selected edge.

12) Return result

Python

```
return mst, total_weight
```

13) Input Section

Python

```
n = int(input("Enter number of vertices: "))
e = int(input("Enter number of edges: "))
```

14) Graph creation

Python

```
graph = {i: [] for i in range(n)}
```

Example:

```
0: []
1: []
2: []
```

15) Input edges

Python

```
u, v, w = map(int, input().split())
```

Example:

```
0 1 5
```

Means:

0 ↔ 1 weight 5

Add edges

Python

```
graph[u].append((v, w))
graph[v].append((u, w))
```

Undirected graph.

16) Run Prim

Python

```
mst, weight = prim_mst(graph)
```

17) Output MST

Python

```
for u, v, w in mst:
    print(f"{u} -- {v} == {w}")
```

18) Print total cost

Python

```
print("Total weight:", weight)
```

19) Example Dry Run

Input:

Vertices = 4
 Edges:
 0 1 10
 0 2 6
 0 3 5
 1 3 15
 2 3 4

Step-by-step:

Start: 0

Heap:

(0,0)

Visit 0:

Push:

(10,1), (6,2), (5,3)

Pick smallest:

(5,3)

From 3:

Push:

(15,1), (4,2)

Pick smallest:

(4,2)

Then:

(10,1)

MST edges:

0 -- 3 == 5
 3 -- 2 == 4
 0 -- 1 == 10

Total = 19

20) AI Concepts Used

- Greedy Search
 - Minimum Spanning Tree
 - Graph Traversal
 - Priority Queue
 - Optimization Problem
 - Local Best Choice Strategy
-

21) Time Complexity

$O(E \log V)$

Because heap operations dominate.

22) Very Important Viva Questions (20)

Basic

1) What is Prim's algorithm?

Greedy algorithm to find MST.

2) What is MST?

Tree with minimum total weight.

3) Data structure used?

Min Heap (Priority Queue)

4) Approach used?

Greedy approach.

5) What is starting node?

Any node to begin MST.

Medium

6) Difference between Prim and Kruskal?

Prim → nodes

Kruskal → edges

7) Why heap is used?

To get minimum weight quickly.

8) Time complexity?

$O(E \log V)$

9) Is graph directed or undirected?

Undirected

10) What is visited array?

Tracks included nodes.

Advanced

11) Why no cycle occurs?

We only add unvisited nodes.

12) What is greedy choice?

Always pick minimum edge.

13) Can we start from any node?

Yes.

14) Does it always give MST?

Yes.

15) Application in AI?

Optimization, clustering, routing.

Expert

16) What is cut property?

Minimum edge crossing cut is chosen.

17) Why better than BFS?

Weighted graph handling.

18) What is heap property?

Smallest element on top.

19) What happens if graph disconnected?

Partial MST (forest)

20) Real world uses?

LAN design, road networks, circuits.

23) 30-Second Viva Answer

Say this:

"This program implements Prim's algorithm to find Minimum Spanning Tree using a greedy approach. It starts from a node and uses a priority queue to always select the minimum weighted edge connecting visited and unvisited nodes. The visited array ensures no cycles. The algorithm continues until all nodes are included and returns the MST with total cost. It is used in network design and optimization problems."

Final Note (Very Important)

You now have:

- ✓ DFS / BFS
- ✓ A* Search
- ✓ Kruskal
- ✓ Prim

👉 This is COMPLETE **AI + Graph Viva set**

If you want next step, I can give:

- 🔥 " Difference table of all 4 algorithms (MOST IMPORTANT FOR VIVA)"
- 🔥 One-page revision sheet
- 🔥 50 rapid-fire viva questions
- 🔥 Common mistakes students make in exams

Just tell 👍



```

☐
def sel():
    arr=list(map(int,input("Enter numbers").split()))
    print("OG: ",arr)
    n =len(arr)
    for i in range(n):
        min = i
        for j in range(i+1,n):

```

```

        if arr[j]<arr[min]:
            min=j
        arr[i],arr[min]=arr[min],arr[i]
    print("Sorted: ",arr)
# sel()
def find(p,i):
    while p[i]!=i:
        i=p[i]
    return i

def uni(p,x,y):
    p[find(p,y)]=find(p,x)

def krusk():
    n=int(input())
    e=int(input())
    edges=[]
    for i in range(e):
        u,v,wt=map(int,input().split())
        edges.append((wt,u,v))
    edges.sort()
    pa=[i for i in range(n)]
    mst=[]
    for wt,u,v in edges:
        if find(pa,u)!=find(pa,v):
            mst.append((u,v,wt))
            uni(pa,u,v)
    tc=0
    for u , v , w in mst:
        tc+=w
    print(f"{u}-->{v}={w}")
    print(tc)
# krusk()

def main():
    flg=0
    while flg==0:
        print("1.Sel_sort\n2.Kruskals\n3.Exit")
        ch=int(input("Enter Your Choice: "))

        if ch==1:
            sel()
        elif ch==2:
            krusk()
        elif ch==3:
            flg=1
        else:
            print("enter valid choice")

    main()

```

This is a **combined AI + DSA practical program** containing:

1. **Selection Sort**
2. **Kruskal's Algorithm (MST)**
3. **Menu-driven system**

Very important for viva because examiners often ask:

👉 "Explain both algorithms in one program"

◆ 1) Overall Purpose of Program

This program:

- Sorts an array using **Selection Sort**
- Finds **Minimum Spanning Tree** using **Kruskal**
- Provides **menu-based execution**
- Repeats until user exits

◆ 2) SELECTION SORT EXPLANATION

Function:

Python
`def sel():`

Sorts array in **ascending order**

Input array:

Python
`arr=list(map(int,input("Enter numbers").split()))`

Example:

5 3 8 1

Logic:

Python
`for i in range(n):
 min = i`

Assume first element is minimum.

Inner loop:

Python

```
for j in range(i+1,n):
```

Search smallest element in remaining array.

Compare:

Python

```
if arr[j] < arr[min]:
    min = j
```

Update index of smallest element.

Swap:

Python

```
arr[i],arr[min]=arr[min],arr[i]
```

Place smallest at correct position.

Example Dry Run:

Input:

5 2 4 1

Steps:

1 2 4 5

Complexity:

$O(n^2)$

◆ 3) KRUSKAL'S ALGORITHM EXPLANATION

FIND function

Python

```
def find(p,i):
    while p[i]!=i:
        i=p[i]
    return i
```

👉 Finds root parent of a node

Used for cycle detection

UNION function

Python

```
def uni(p,x,y):
    p[find(p,y)] = find(p,x)
```

👉 Merges two sets

KRUSKAL FUNCTION

Python

```
def krusk():
```

Finds **Minimum Spanning Tree**

Input:

Python

```
n=int(input())
e=int(input())
```

- n = vertices
 - e = edges
-

Edge input:

Python

```
u,v,wt=map(int,input().split())
edges.append((wt,u,v))
```

Stored as:

👉 (weight, u, v)

Sorting edges:

Python

```
edges.sort()
```

Smallest weight first (Greedy step)

Parent array:

Python

```
pa=[i for i in range(n)]
```

Each node is its own parent initially

MST creation:

Python

```
if find(pa,u)!=find(pa,v):
```

If not in same set → no cycle

Add edge:

Python

```
mst.append((u,v,wt))
uni(pa,u,v)
```

Total cost:

Python

```
tc+=w
```

Output:

Example:

```
0-->1=2
1-->2=3
Total=5
```

Complexity:

$O(E \log E)$

◆ 4) MENU DRIVEN PROGRAM

Python

```
while flg==0:
```

Runs until exit.

Options:

Python

1. Selection Sort
2. Kruskal
3. Exit

Choice handling:

Python

```
if ch==1:
    sel()
```

Runs selection sort

Python

```
elif ch==2:
    krusk()
```

Runs Kruskal

Python

```
elif ch==3:
    flg=1
```

Exit program

◆ 5) IMPORTANT AI + DSA CONCEPTS

Selection Sort:

- Sorting algorithm
 - Brute force approach
 - Not AI but basic DSA
-

Kruskal:

- Greedy algorithm
 - Optimization problem
 - Used in AI graph search
-

Common Concepts:

- Graph theory
- Disjoint Set (Union-Find)

- Greedy approach
 - Optimization
 - Sorting
-

◆ 6) 20 VIVA QUESTIONS (VERY IMPORTANT)

◆ Selection Sort

1) What is selection sort?

Sorting by selecting minimum element repeatedly.

2) Time complexity?

$O(n^2)$

3) Is it stable?

No

4) Space complexity?

$O(1)$

5) Where used?

Small datasets

◆ Kruskal

6) What is Kruskal algorithm?

Greedy MST algorithm.

7) What is MST?

Minimum cost spanning tree.

8) Why sorting edges?

To apply greedy strategy.

9) What is Union-Find?

Structure to detect cycles.

10) Why cycle check needed?

To avoid loops in MST.

◆ Combined Questions

11) Which algorithm is greedy?

Kruskal

12) Selection sort type?

Brute force

13) Kruskal complexity?

$O(E \log E)$

14) Selection sort complexity?

$O(n^2)$

15) What is parent array?

Stores component roots

◆ Advanced Viva

16) Why Kruskal is used in AI?

For optimization problems.

17) Difference between Kruskal and Prim?

Kruskal → edges, Prim → nodes

18) Why union-find used?

Efficient cycle detection.

19) What is greedy choice?

Locally optimal decision.

20) Real applications?

Networks, routing, clustering

◆ 7) 30-SECOND VIVA ANSWER

Say this:

“This program is a menu-driven implementation of Selection Sort and Kruskal’s algorithm. Selection sort sorts an array by repeatedly selecting the minimum element. Kruskal’s

algorithm is a greedy algorithm used to find Minimum Spanning Tree using Union-Find to avoid cycles. The program allows user to choose operation and displays sorted array or MST with total cost."

FINAL NOTE

This single program covers:

- ✓ Sorting (Selection Sort)
- ✓ Graph Algorithm (Kruskal MST)
- ✓ Greedy technique
- ✓ Menu-driven logic
- ✓ Union-Find structure

If you want next level preparation, I can give:

- ✓ One-page viva revision sheet (ALL ALGORITHMS)
- ✓ Comparison table: BFS vs DFS vs A* vs Prim vs Kruskal
- ✓ Top 50 viva questions (guaranteed exam set)

Just tell 



```

□
def sort(nums):
    for i in range (len(nums)-1):
        minpos=i
        for j in range (i,len(nums)):
            if nums[j]<nums[minpos]:
                minpos=j
        #swapping

        temp=nums[i]
        nums[i]=nums[minpos]
        nums[minpos]=temp

    print(nums)

user_ip=input("enter the numbers ")
nums=[int (x) for x in user_ip.split()]

sort(nums)
print("sorted list : ",nums)

```

This is a **Selection Sort program** (same algorithm as earlier, just written in a simpler style with step-by-step output).

I'll explain it in **viva-friendly format**.

◆ 1) Purpose of Program

This program:

- Takes a list of numbers from user
 - Sorts them in **ascending order**
 - Uses **Selection Sort algorithm**
 - Prints array after each pass (step-by-step sorting)
-

◆ 2) Core Idea of Selection Sort

Selection sort works like:

- 👉 Find smallest element
 - 👉 Put it at correct position
 - 👉 Repeat for remaining array
-

◆ 3) Code Explanation

◆ Function definition

Python

```
def sort(nums):
```

This function sorts the list `nums`.

◆ Outer loop

Python

```
for i in range(len(nums)-1):
```

- Runs from 0 to n-2
 - Each iteration places correct element at position `i`
-

◆ Assume minimum position

Python

```
minpos = i
```

Assume current index has smallest element.

◆ Inner loop

Python

```
for j in range(i, len(nums)):
```

Searches smallest element in remaining array.

◆ Comparison

Python

```
if nums[j] < nums[minpos]:
    minpos = j
```

If smaller element found → update index.

◆ Swapping

Python

```
temp = nums[i]
nums[i] = nums[minpos]
nums[minpos] = temp
```

This places smallest element in correct position.

◆ 4) Step-by-step output

Python

```
print(nums)
```

Prints array after each pass → shows sorting process clearly.

◆ 5) Input handling

Python

```
user_ip = input("enter the numbers ")
```

User enters numbers in one line.

Example:

5 3 1 4

Python

```
nums = [int(x) for x in user_ip.split()]
```

Converts input into list:

Python

```
[5, 3, 1, 4]
```

◆ 6) Function call

Python

```
sort(nums)
```

Sorts the list.

Python

```
print("sorted list : ", nums)
```

Final sorted output.

◆ 7) Example Dry Run

Input:

5 2 4 1

Pass 1:

Smallest = 1 → swap with 5

1 2 4 5

Pass 2:

Already correct

1 2 4 5

Pass 3:

Already correct

◆ 8) Complexity

$O(n^2)$

◆ 9) Key Viva Concepts

- Sorting Algorithm
 - Selection Sort
 - Brute Force Approach
 - In-place sorting
 - Swapping elements
 - Time complexity
-

◆ 10) 20 Viva Questions + Answers

◆ Basic

1) What is selection sort?

Sorting by selecting minimum element repeatedly.

2) What is time complexity?

$O(n^2)$

3) Is it stable?

No

4) Is it in-place?

Yes

5) How many loops used?

Two nested loops

◆ Medium

6) What does minpos store?

Index of smallest element

7) Why swapping is needed?

To place element in correct position

8) Best case complexity?

$O(n^2)$

9) Worst case complexity?

$O(n^2)$

10) Space complexity?

$O(1)$

◆ Advanced

11) Why selection sort is not efficient?

Because of repeated comparisons

12) Where is it used?

Small datasets, teaching purpose

13) Is recursion used?

No

14) Is it adaptive?

No

15) Does it preserve order?

No (not stable)

◆ Expert

16) Why called selection sort?

Because it selects minimum each time

17) What happens each pass?

One element gets fixed

18) Difference from bubble sort?

Selection picks min, bubble swaps adjacent

19) Why print inside loop?

To show step-by-step sorting

20) Real life use?

Educational purposes only

◆ 11) 30-Second Viva Answer

Say this:

“This program implements Selection Sort algorithm. It repeatedly finds the minimum element from unsorted part and swaps it with the first unsorted position. The process continues until the array is sorted. It uses two loops and has time complexity $O(n^2)$. It is an in-place and simple sorting algorithm used mainly for learning purposes.”

🔥 FINAL NOTE

You now have:

- ✓ Selection Sort (step-wise + simple version)
- ✓ Kruskal MST
- ✓ Prim MST
- ✓ A* Search
- ✓ DFS/BFS

👉 This is your COMPLETE AI + DSA VIVA SET.

If you want next, I can give:

- ✓ "ALL ALGORITHMS comparison table (SUPER IMPORTANT)"
- ✓ "Top 50 viva questions repeated in exams"
- ✓ "1-page revision cheat sheet for exam day"

Just tell 👍



```

☐
def issafe(arr, x, y, n):
    for row in range(x):
        if arr[row][y] == 1:
            return False

    row = x
    col = y
    while row >= 0 and col >= 0:
        if arr[row][col] == 1:
            return False
        row -= 1
        col -= 1

    row = x
    col = y
    while row >= 0 and col < n:
        if arr[row][col] == 1:
            return False
        row -= 1
        col += 1

    return True

def nQueen(arr, x, n):
    if x >= n:
        return True
  
```

```

for col in range(n):
    if issafe(arr, x, col, n):
        arr[x][col] = 1
        if nQueen(arr, x + 1, n):
            return True
        arr[x][col] = 0

return False

def main():
    n = int(input("Enter number of Queens : "))
    arr = [[0] * n for _ in range(n)]

    if nQueen(arr, 0, n):
        for i in range(n):
            for j in range(n):
                if arr[i][j] == 1:
                    print("Q", end=" ")
                else:
                    print(".", end=" ")
            print()

if __name__ == '__main__':
    main()

```

This is the **N-Queens Problem**, one of the most important **Backtracking problems in AI (Artificial Intelligence)** and DSA.

It is a **very high-frequency viva question**.

◆ 1) Problem Statement

Place **N queens on an N×N chessboard** such that:

- ✓ No two queens attack each other
- ✓ No same row
- ✓ No same column
- ✓ No same diagonal

◆ 2) AI Concept Used

This is solved using:

👉 **Backtracking Search**

Meaning:

- Try a solution
- If it fails → undo (backtrack)
- Try next option

◆ 3) Code Explanation

◆ Safety Check Function

Python

```
def issafe(arr, x, y, n):
```

Checks whether placing queen at (x, y) is safe.

✓ Check column

Python

```
for row in range(x):
    if arr[row][y] == 1:
        return False
```

👉 Checks if any queen already in same column.

✓ Check left diagonal

Python

```
while row >= 0 and col >= 0:
```

Moves diagonally ↖

If queen found → unsafe.

✓ Check right diagonal

Python

```
while row >= 0 and col < n:
```

Moves diagonally ↗

If queen found → unsafe.

✓ If safe

Python

```
return True
```

Position is valid.

◆ 4) Main Backtracking Function

Python

```
def nQueen(arr, x, n):
```

- x = current row
 - n = board size
-

Base condition

Python

```
if x >= n:
    return True
```

✓ All queens placed successfully

Try all columns

Python

```
for col in range(n):
```

Try placing queen in each column of row x.

Check safe position

Python

```
if issafe(arr, x, col, n):
```

If safe → place queen.

Place queen

Python

```
arr[x][col] = 1
```

Recursive call

Python

```
if nQueen(arr, x + 1, n):
    return True
```

Move to next row.

Backtracking step

Python

```
arr[x][col] = 0
```

If solution fails → remove queen.

If no position works

Python

```
return False
```

◆ 5) Main Function

Python

```
n = int(input("Enter number of Queens : "))
```

User input N.

Create board

Python

```
arr = [[0] * n for _ in range(n)]
```

Example for n=4:

```
0 0 0 0
0 0 0 0
0 0 0 0
0 0 0 0
```

Solve problem

Python

```
if nQueen(arr, 0, n):
```

Start from row 0.

Print solution

Python

```
print("Q", end=" ")
```

Q = Queen
 . = empty space

◆ 6) Example Output (N = 4)

```
. Q . .
. . . Q
Q . . .
. . Q .
```

◆ 7) AI Concept (VERY IMPORTANT)

This is:

👉 Backtracking Search Algorithm

Used in:

- Constraint Satisfaction Problems (CSP)
- Puzzle solving
- Sudoku
- Graph coloring
- Combinatorial problems

◆ 8) Why Backtracking Works

Steps:

1. Try placing queen
2. Check constraints
3. If valid → move forward
4. If not → undo (backtrack)

◆ 9) Complexity

Worst case:

$O(N!)$

Because we try many combinations.

◆ 10) Key Viva Concepts

- Backtracking
- Constraint Satisfaction Problem (CSP)

- Recursion
 - Safe position checking
 - Pruning
 - State space tree
-

◆ 11) 20 Viva Questions + Answers

◆ Basic

1) What is N-Queens problem?

Place N queens without attacking each other.

2) Which technique is used?

Backtracking.

3) What is backtracking?

Trying solutions and undoing invalid ones.

4) What is base condition?

All queens placed ($x \geq n$)

5) What is output?

Valid board configuration

◆ Medium

6) Why diagonals checked?

Queens attack diagonally

7) Why recursion used?

To try row by row placement

8) Time complexity?

$O(N!)$

9) Space complexity?

$O(N^2)$

10) What is issafe function?

Checks valid placement

◆ Advanced

11) What is CSP?

Constraint Satisfaction Problem

12) Why backtracking needed?

To remove invalid paths

13) What happens if no solution?

Returns False

14) Is greedy used?

No

15) Is DP used?

No

◆ Expert

16) Why row-wise placement?

To reduce complexity

17) What is state space tree?

All possible board states

18) Why pruning used?

To remove invalid paths early

19) Real applications?

AI, scheduling, puzzle solving

20) Is solution unique?

No, multiple solutions possible

◆ 12) 30-Second Viva Answer

Say this:

"This program solves the N-Queens problem using backtracking. It places N queens on an N×N chessboard such that no two queens attack each other. It checks row, column, and diagonal safety before placing a queen. If a conflict occurs, it backtracks and tries another position. It is a classic constraint satisfaction problem used in AI search techniques."

FINAL SUMMARY

You now have FULL coverage of:

- ✓ DFS / BFS
- ✓ A* Search
- ✓ Kruskal
- ✓ Prim
- ✓ Selection Sort
- ✓ N-Queens (Backtracking)

If you want next, I can give:

-  " Complete AI Viva Cheat Sheet (ALL ALGORITHMS in 2 pages)"
-  " Most repeated 50 viva questions (guaranteed exam set)"
-  " Difference table of all search algorithms (VERY IMPORTANT)"

Just tell 



```

☐
def is_safe(b, r, c):
    for i in range(r):
        if b[i] == c or abs(b[i]-c) == r-i:
            return False
    return True

def solve(b, r, n):
    if r == n:
        return b
    for c in range(n):
        if is_safe(b, r, c):
            b[r] = c
            res = solve(b, r+1, n)
            if res: return res
    return None

def print_board(b, n):
    for i in range(n):
        print(' '.join(' Q ' if j == b[i] else '.' for j in range(n)))

n = 4

```

```
board = [-1]*n
sol = solve(board, 0, n)
if sol:
    print_board(sol, n)
```

This is another **N-Queens problem implementation**, but this version is more optimized and clean. It is a **classic Backtracking + Constraint Satisfaction Problem (CSP)** used in **AI search techniques**.

◆ 1) Purpose of Program

This program:

- Solves **N-Queens problem**
- Uses **backtracking**
- Stores solution in **1D array (optimized approach)**
- Prints final chessboard configuration

◆ 2) Key Idea (Important for Viva)

Instead of using a 2D matrix, this code uses:

👉 **b[r] = c**

Meaning:

- **r** = row
- **c** = column
- **b[r]** stores queen position in row **r**

Example:

b = [1, 3, 0, 2]

Means:

- Row 0 → Column 1
- Row 1 → Column 3
- Row 2 → Column 0
- Row 3 → Column 2

◆ 3) Code Explanation

◆ Safety Check Function

Python

```
def is_safe(b, r, c):
```

Checks if queen can be placed at row r , column c .

✓ Column check

Python

```
if b[i] == c
```

👉 Same column conflict

✓ Diagonal check

Python

```
abs(b[i]-c) == r-i
```

👉 Checks diagonal attack condition

Why?

If distance in row == distance in column → diagonal collision

✓ If unsafe

Python

```
return False
```

✓ Safe position

Python

```
return True
```

◆ 4) Main Solve Function

Python

```
def solve(b, r, n):
```

- b = board (1D array)
 - r = current row
 - n = size of board
-

Base condition

Python

```
if r == n:
    return b
```

✓ All queens placed successfully

Try all columns

Python

```
for c in range(n):
```

Try each column in row r.

Check safety

Python

```
if is_safe(b, r, c):
```

If valid position → place queen.

Place queen

Python

```
b[r] = c
```

Recursive call

Python

```
res = solve(b, r+1, n)
```

Move to next row.

If solution found

Python

```
if res: return res
```

Stop early (first valid solution only).

Backtrack case

Python

```
return None
```

If no solution possible.

◆ 5) Print Board Function

Python

```
def print_board(b, n):
```

Displays final chessboard.

Logic:

Python

```
' Q ' if j == b[i] else '.'
```

- If queen exists → Q
- Else → .

Example Output (n=4)

```
. Q . .
. . . Q
Q . . .
. . Q .
```

◆ 6) Initialization

Python

```
n = 4
board = [-1]*n
```

Board starts empty:

```
[-1, -1, -1, -1]
```

◆ 7) Solve call

Python

```
sol = solve(board, 0, n)
```

Start from row 0.

◆ 8) Print solution

Python

```
if sol:
    print_board(sol, n)
```

Displays final arrangement.

◆ 9) AI Concepts Used

This is a **classic AI search problem**:

- ✓ Backtracking
- ✓ Constraint Satisfaction Problem (CSP)
- ✓ State Space Search
- ✓ Depth-first search idea
- ✓ Pruning invalid states

◆ 10) Why This Version is Better

Compared to earlier 2D version:

Feature	Old Version	This Version
Storage	2D array	1D array
Speed	Slower	Faster
Memory	More	Less
Logic	Complex	Simple

◆ 11) Complexity

Worst case:

$O(N!)$

◆ 12) Important Viva Questions (20)

◆ Basic

1) What is N-Queens problem?

Place N queens so that no two attack each other.

2) Which technique is used?

Backtracking.

3) Why backtracking?

To undo invalid solutions.

4) What is base condition?

$r == n$

5) What is board representation?

1D array storing column positions.

◆ Medium

6) Why 1D array used?

Efficient representation.

7) What does $b[r]$ represent?

Column position of queen in row r .

8) What is diagonal condition?

$\text{abs}(b[i]-c) == r-i$

9) Time complexity?

$O(N!)$

10) Is it DFS or BFS?

DFS (backtracking style)

◆ Advanced

11) What is CSP?

Constraint Satisfaction Problem

12) Why pruning is used?

To eliminate invalid states early

13) Does it give all solutions?

No, only first solution

14) What is state space?

All possible board configurations

15) Is greedy used?

No

◆ Expert

16) Why diagonal check works?

Because queens move diagonally in chess

17) What is optimization here?

1D array reduces memory

18) Can multiple solutions exist?

Yes

19) Application in AI?

Scheduling, CSP problems, puzzles

20) Is recursion necessary?

Yes, for backtracking

◆ 13) 30-Second Viva Answer

Say this:

"This program solves the N-Queens problem using backtracking. It uses a 1D array where each index represents a row and value represents column position of queen. It checks column and diagonal safety before placing a queen. If a conflict occurs, it backtracks. It is a constraint satisfaction problem widely used in AI search techniques."

🔥 FINAL SUMMARY (VERY IMPORTANT)

You now have FULL coverage of:

- ✓ Selection Sort
 - ✓ Kruskal
 - ✓ Prim
 - ✓ A* Search
 - ✓ DFS/BFS
 - ✓ N-Queens (2 versions)
-

If you want next step, I can give:

- 🔥 "ONE SHEET AI VIVA REVISION (ALL ALGORITHMS + FORMULAS)"
- 🔥 "Top 50 MOST ASKED QUESTIONS in exams"
- 🔥 "Difference table: Backtracking vs Greedy vs Search algorithms"

Just tell 👍



import time

```
def greet_user():
    print("Hello! Welcome to our store. How can I assist you today?")
    time.sleep(1)

def handle_query(query):
    query = query.lower()

    if "store hours" in query or "open hours" in query:
        return "Our store is open from 9 AM to 9 PM, Monday to Saturday."
    elif "products" in query or "catalog" in query:
        return "We offer a wide range of products including electronics, clothing, and home goods. What would you like to know about?"
    elif "location" in query or "address" in query:
        return "We are located at 123 Main Street, YourCity, YourState."
    elif "contact" in query or "support" in query:
        return "You can contact us at support@ourstore.com or call us at (123) 456-7890."
    elif "thank you" in query or "bye" in query:
        return "You're welcome! Feel free to reach out if you have any more questions. Goodbye!"
    else:
        return "Sorry, I didn't quite understand that. Can you please rephrase?"

def start_chat():
    greet_user()

    while True:
        user_input = input("\nYou: ")

        if "bye" in user_input.lower() or "exit" in user_input.lower():
            print("Bot: Goodbye! Have a great day!")
            break

        response = handle_query(user_input)
        print("Bot:", response)

# Start the chatbot interaction
start_chat()
```

This is a **rule-based chatbot program** used in AI (Artificial Intelligence), specifically under:

👉 **Natural Language Processing (NLP) + Expert System (Simple form)**

It is very important for viva because it shows how a **basic AI chatbot works without machine learning**.

◆ 1) Purpose of Program

This program:

- Acts as a **customer support chatbot**
 - Takes user input (text)
 - Matches keywords
 - Gives predefined responses
 - Runs continuously until user exits
-

◆ 2) AI Concept Used

This is:

👉 **Rule-Based AI System**

Meaning:

- No training data
- No ML model
- Only IF-ELSE rules
- Keyword matching

Used in:

- Simple chatbots
 - FAQ bots
 - Helpdesk systems
-

◆ 3) Code Explanation

◆ Import time module

Python

```
import time
```

Used for delay (to make chatbot realistic).

◆ Greeting function

Python

```
def greet_user():
```

Welcomes the user.

Python

```
print("Hello! Welcome to our store. How can I assist you today?")
time.sleep(1)
```

- Prints greeting
- Waits 1 second

👉 Makes chatbot feel human-like

◆ 4) Main Logic Function

Python

```
def handle_query(query):
```

This function processes user input.

Convert to lowercase

Python

```
query = query.lower()
```

👉 Makes matching case-insensitive

Example:

"Location" → "location"

◆ 5) Rule-Based Conditions

● Store hours

Python

```
if "store hours" in query
```

Response:

👉 9 AM to 9 PM

● Products

Python

```
elif "products" in query
```

Gives product info.

Location

Python

```
elif "location" in query
```

Returns address.

Contact

Python

```
elif "contact" in query
```

Returns email + phone.

Goodbye condition

Python

```
elif "thank you" in query or "bye" in query
```

Polite response.

Default case

Python

```
else:
    return "Sorry, I didn't understand"
```

If no rule matches.

6) Chat Loop Function

Python

```
def start_chat():
```

Starts chatbot session.

Greeting call

Python

```
greet_user()
```

Infinite loop

Python

```
while True:
```

Chat continues until user exits.

Take input

Python

```
user_input = input("\nYou: ")
```

Exit condition

Python

```
if "bye" in user_input.lower() or "exit" in user_input.lower():
```

Stops chatbot.

Print response

Python

```
print("Bot:", response)
```

◆ 7) Program Start

Python

```
start_chat()
```

Begins chatbot execution.

◆ 8) Example Interaction

Hello! Welcome to our store. How can I assist you today?

You: what are your store hours

Bot: Our store is open from 9 AM to 9 PM

You: location

Bot: We are located at 123 Main Street

You: bye

Bot: Goodbye! Have a great day!

◆ 9) AI Concepts Used

- ✓ Rule-Based System
 - ✓ NLP (Keyword matching)
 - ✓ Expert System
 - ✓ Pattern matching
 - ✓ State-less conversation
-

◆ 10) Limitations (VERY IMPORTANT VIVA POINT)

- Cannot understand natural language properly
 - No learning capability
 - Fixed responses only
 - No memory of past conversation
 - Not context-aware
-

◆ 11) Advantages

- Simple
 - Fast
 - Easy to implement
 - Good for FAQs
-

◆ 12) Complexity

$O(1)$ per query

◆ 13) 20 Viva Questions + Answers

◆ Basic

1) What is this program?

Rule-based chatbot system.

2) What is NLP?

Processing human language using AI.

3) Is ML used here?

No.

4) What is rule-based system?

System using IF-ELSE conditions.

5) Why lower() used?

To make input case-insensitive.

◆ Medium

6) What is chatbot?

AI system that interacts with humans.

7) What is intent matching?

Matching keywords in input.

8) Does chatbot learn?

No.

9) What is fallback response?

Default "I didn't understand".

10) Why time.sleep used?

To simulate real human delay.

◆ Advanced

11) What is expert system?

AI using fixed rules.

12) Can it handle complex sentences?

No.

13) What is limitation?

No intelligence or learning.

14) Where used?

Customer support FAQs.

15) Is it scalable?

Not much.

◆ Expert

16) Difference from AI chatbot like ChatGPT?

ChatGPT uses ML + NLP, this uses rules only.

17) What is state of chatbot?

Stateless (no memory)

18) What is keyword matching?

Searching words in input string.

19) Can we improve it?

Yes using NLP/ML models.

20) Type of AI?

Weak AI (Narrow AI)

◆ 14) 30-Second Viva Answer

Say this:

"This program implements a rule-based chatbot using Python. It uses simple keyword matching to respond to user queries like store hours, location, and contact details. It does not use machine learning; instead it relies on predefined if-else rules. It is an example of a simple expert system used in basic NLP applications."

🔥 FINAL SUMMARY

You now have FULL AI practical coverage:

- ✓ Search Algorithms (BFS, DFS, A*)
- ✓ Greedy Algorithms (Prim, Kruskal)
- ✓ Backtracking (N-Queens)
- ✓ Sorting (Selection Sort)
- ✓ NLP Chatbot (Rule-based AI)

If you want next step, I can give:

- 🔥 " Complete AI Viva Cheat Sheet (ALL PROGRAMS ONE PAGE)"
- 🔥 " Top 50 repeated viva questions (GUARANTEED EXAM SET)"
- 🔥 " DIFFERENCE TABLE (ALL ALGORITHMS COMBINED)"

Just tell 👍



```
☐
def evaluate_employee():
    print("Welcome to the Employee Performance Evaluation System!")
```



```
# Get inputs from the user
try:
    work_quality = int(input("\nRate the work quality (1-5): "))
    attendance = int(input("Enter the number of days present out of 30: "))
    teamwork = int(input("Rate teamwork and collaboration (1-5): "))

    # Validate inputs
    if work_quality < 1 or work_quality > 5 or teamwork < 1 or teamwork > 5:
        print("\nPlease enter ratings between 1 and 5 for Work Quality and Teamwork.")
        return

    if attendance < 0 or attendance > 30:
        print("\nPlease enter a valid attendance number (0 to 30 days).")
        return

except ValueError:
    print("\nInvalid input. Please enter numeric values.")
    return

# Calculate performance score
performance_score = (work_quality + teamwork + (attendance / 30) * 5) / 3

# Evaluate performance
if performance_score >= 4.5:
    evaluation = "Excellent"
    feedback = "Outstanding work. Keep up the great performance!"
elif performance_score >= 3.5:
    evaluation = "Good"
    feedback = "Good job! There is room for improvement in some areas."
elif performance_score >= 2.5:
    evaluation = "Needs Improvement"
    feedback = "Performance is below expectations. Focus on improvement."
else:
    evaluation = "Poor"
    feedback = "Performance is unsatisfactory. Consider taking corrective actions."

# Output evaluation result
print(f"\nPerformance Evaluation: {evaluation}")
print(f"Feedback: {feedback}")

# Run the system
evaluate_employee()
```

This is a **simple AI-based Expert System (Rule-Based Decision System)** used for **Employee Performance Evaluation**.

It is very important for viva because it shows how AI systems make **decisions using rules + scoring logic**.

◆ 1) Purpose of Program

This program:

- Takes employee inputs:
 - Work quality
 - Attendance
 - Teamwork
 - Calculates performance score
 - Classifies employee as:
 - Excellent
 - Good
 - Needs Improvement
 - Poor
 - Gives feedback
-

◆ 2) AI Concept Used

👉 This is a **Rule-Based Expert System**

Meaning:

- No machine learning
- No training data
- Decision based on fixed rules
- Uses **IF-ELSE logic**

Used in:

- HR evaluation systems
 - Basic AI decision systems
 - Expert systems in AI
-

◆ 3) Code Explanation

◆ Function start

Python

```
def evaluate_employee():
```

Main function for evaluation system.

◆ Welcome message

Python

```
print("Welcome to the Employee Performance Evaluation System!")
```

Simple UI message.

◆ 4) Input Section

Python

```
work_quality = int(input("Rate the work quality (1-5): "))
```

Employee rating from 1 to 5.

Python

```
attendance = int(input("Enter the number of days present out of 30: "))
```

Attendance out of 30 days.

Python

```
teamwork = int(input("Rate teamwork (1-5): "))
```

Teamwork score.

◆ 5) Input Validation

Check rating range

Python

```
if work_quality < 1 or work_quality > 5
```

Ensures valid rating.

Check attendance range

Python

```
if attendance < 0 or attendance > 30:
```

Ensures realistic value.

Exception handling

Python

except ValueError:

Handles non-numeric input.

Example:

👉 "abc" → invalid input

◆ 6) Performance Calculation

Python

```
performance_score = (work_quality + teamwork + (attendance / 30) * 5) / 3
```

Breakdown:

- Work quality (1-5)
- Teamwork (1-5)
- Attendance normalized to 5 scale

Attendance normalization:

$$\frac{\text{attendance}}{30} \times 5$$

👉 Converts attendance to 5-point scale

Final formula:

$$\frac{\text{work_quality} + \text{teamwork} + \text{attendance_score}}{3}$$

👉 Average of all factors

◆ 7) Decision Logic (Expert System Rules)

★ Excellent

Python

```
if performance_score >= 4.5:
```

High performance employee.

👍 Good

Python

```
elif performance_score >= 3.5:
```

Printed using [ChatGPT to PDF](#), powered by PDFCrowd [HTML to PDF API](#).

Above average.

Needs Improvement

Python

```
elif performance_score >= 2.5:
```

Below expectations.

Poor

Python

```
else:
```

Low performance.

8) Output Section

Python

```
print(f"\nPerformance Evaluation: {evaluation}")
print(f"Feedback: {feedback}")
```

Displays result + feedback.

9) Example Output

Input:

```
Work quality = 5
Attendance = 28
Teamwork = 4
```

Output:

```
Performance Evaluation: Excellent
Feedback: Outstanding work. Keep it up!
```

10) AI Concepts Used

- ✓ Expert System
 - ✓ Rule-Based Decision Making
 - ✓ Scoring System
 - ✓ Classification
 - ✓ Human-like reasoning simulation
-

11) Advantages

- Simple logic
 - Easy to implement
 - Fast decision making
 - Used in HR systems
-

◆ 12) Limitations

- No learning ability
 - Fixed rules only
 - Not adaptive
 - No real intelligence
-

◆ 13) Time Complexity

$O(1)$

👉 Constant time (only calculations + conditions)

◆ 14) 20 Viva Questions + Answers

◆ Basic

1) What is this program?

Employee performance evaluation system.

2) What type of AI system is used?

Rule-based expert system.

3) Is machine learning used?

No.

4) What inputs are taken?

Work quality, attendance, teamwork.

5) What is output?

Performance rating + feedback.

◆ Medium

6) What is performance score?

Average of all parameters.

7) Why attendance is divided by 30?

To normalize to 5 scale.

8) What is exception handling?

Handling invalid input.

9) What happens if input is wrong?

Error message is shown.

10) What is rule-based system?

System using IF-ELSE rules.

◆ Advanced

11) What is expert system?

AI system using predefined knowledge.

12) What is classification here?

Excellent / Good / Poor categories.

13) Why scoring is used?

To quantify performance.

14) Can system learn?

No.

15) Is it adaptive?

No.

◆ Expert

16) Where is it used?

HR evaluation systems.

17) Why normalization needed?

To make values comparable.

18) What type of AI?

Weak AI

19) Can we improve it?

Yes using ML models.

20) Real-world use?

Employee appraisal systems

 15) 30-Second Viva Answer

Say this:

"This program is a rule-based expert system for employee performance evaluation. It takes inputs like work quality, attendance, and teamwork, calculates a normalized performance score, and classifies employees into categories like Excellent, Good, or Poor using predefined rules. It does not use machine learning and is an example of decision-making in AI systems."

 FINAL SUMMARY (YOUR COMPLETE AI VIVA SET)

You now have FULL coverage of:

- ✓ Search Algorithms (DFS, BFS, A*)
 - ✓ Greedy Algorithms (Prim, Kruskal)
 - ✓ Backtracking (N-Queens)
 - ✓ Sorting (Selection Sort)
 - ✓ NLP Chatbot
 - ✓ Expert System (Employee Evaluation)
-

If you want next, I can give:

-  "ONE PAGE FINAL AI CHEAT SHEET (ALL PROGRAMS + FORMULAS)"
-  "TOP 50 MOST IMPORTANT VIVA QUESTIONS (GUARANTEED)"
-  "DIFFERENCE TABLE (EVERY ALGORITHM COMPARISON)"

Just tell 👍